

Design And Analysis of Algorithm
Complex Computing Problem [Recurrence Solver]

Section: A



Submitted By:

Syed Shayan Arshad	2023-CS-625
Adeel-Ur-Rehman	2023-CS-626
Hammad Khalid	2023-CS-637

Submitted To:

Mam. Drakhshan Bokhat

Dated: 14th April 2025

Department of Computer Science
University of Engineering and Technology Lahore, New Campus

Complex Recurrence Solver

Objective

To implement a comprehensive C++ program that solves various types of recurrence relations commonly encountered in algorithm analysis, using:

- Master Theorem
- Extended Master Theorem
- Decreasing Recurrence Method
- Multi-way Recurrence Approximation Method

Flow of System

User Input:

The program presents a menu with three recurrence relation types:

- $T(n) = aT(n-b) + f(n)$ [Decreasing Recurrence]
- $T(n) = aT(n/b) + f(n)$ [Standard Divide & Conquer]
- $T(n) = aT(n/b) + a'T(n/b') + f(n)$ [Multi-way Divide & Conquer]

Based on selection, the program collects appropriate parameters (a, b, and f(n))

For f(n), users can specify polynomial terms (n^k), logarithmic terms ($\log^j$), or combinations

Parser Module:

- The system parses f(n) using string parsing techniques to extract:
 - Polynomial degree (e.g., from n^2 , extracts 2)
 - Logarithmic exponent (e.g., from $\log^3$, extracts 3)
- Two key functions handle this analysis:
 - `extractLogExponent()`: Identifies logarithmic terms and their powers
 - `parseFunction()`: Returns a pair containing polynomial degree and logarithmic exponent

Method Identification and Selection

- Standard recurrences with polynomial f(n): Uses Master Theorem
- Standard recurrences with logarithmic terms: Uses Extended Master Theorem
- Decreasing recurrences: Uses specialized analysis for $T(n) = aT(n-b) + f(n)$ format
- Multi-way recurrences: Offers three approximation strategies:
 - Shallow analysis (lower bound)
 - Deep analysis (upper bound)
 - Average approach (combined parameters)

Solution Module

The system applies appropriate theorems based on recurrence type and function characteristics. Each method compares growth rates between recursive and non-recursive parts to determine the dominant term.

Output:

Presents detailed step-by-step analysis of the recurrence

Shows which case of which theorem was applied

Explains how parameters influence the final complexity

Returns asymptotic complexity in appropriate notation (Θ , O , or Ω)

Technology:

Language: C++

Libraries: iostream, cmath, string, iomanip